causal forest

2026-09-30

Contents

1 Setup and data preparation **2**

1.1 Downloading required packages . . . 2

1.2 Loading data . . . 2

1.3 Forming the pretreatment covariate variables and moderators . . . 2

1.4 Setting our ex-ante moderators . . . 3

1.5 Data transformation . . . 3

2 Diagnostic tests **3**

2.1 Missing values across waves . . . 4

2.2 Statistics about the data . . . 4

2.3 Name penalty by wave . . . 6

2.4 Overlap/Propensity check . . . 6

3 Causal forests **10**

3.1 Specifications . . . 10

3.2 Fit . . . 10

4 Causal forest results **11**

4.1 Average treatment effect . . . 11

4.2 Estimated effects (CATE) . . . 12

4.3 Subgroup plots . . . 16

5 Validation of causal forest results **21**

5.1 test_calibration . . . 21

5.2 RATE / TOC with a firm-level split . . . 22

5.3 Combined reading (rule fixed in advance) . . . 25

5.4 Tuning robustness . . . 26

This document runs in four stages: **diagnostic tests** on the sample, **causal forests** (fit and overlap check), **causal forest results** (ATE, estimated effects, moderator table, best linear projection) and **validation** of those results (calibration, RATE/TOC, combined verdict, tuning robustness). Covariate balance does not depend on any forest, so it lives in a separate file: `Causal Forest Balance Table.Rmd`. Both files read the analysis sample saved by `cleaned_data.Rmd` (`cleaned data.Rda`), so knit that one first.

1 Setup and data preparation

1.1 Downloading required packages

```
required_packages <- c("estimatr", "dplyr", "tidyr", "grf", "knitr")
missing_packages  <- setdiff(required_packages, rownames(installed.packages()))
if (length(missing_packages) > 0) {
  install.packages(missing_packages, repos = "https://cloud.r-project.org")
}
library(estimatr)
library(ggplot2)
library(dplyr)
library(tidyr)
library(grf)

library(knitr)
```

1.2 Loading data

Obtaining cleaned_data.Rda from running cleaned_data.rmd

```
# cleaned_data.Rmd builds and saves the analysis sample (df_int1)
rda_path <- "cleaned data.Rda"
if (!file.exists(rda_path)) {
  stop("Cleaned data not found. Knit cleaned_data.Rmd first to create: ",
       normalizePath(rda_path, mustWork = FALSE))
}
loaded <- load(rda_path)
stopifnot("df_int1" %in% loaded)
```

1.3 Forming the pretreatment covariate variables and moderators

This selects pre-treatment covariates that may explain heterogeneity in treatment effects

```
all_pretreat_cov <- c(
  "language_skills",
  "female",
  "ba_quality",
  "extracurricular_skills",
  "ma",
  "same_exp",
  "exp_highquality"
  )
# df_int1 comes from cleaned_data.Rmd: Canadian-named (control) and Chinese/Indian/
# Pakistani-named (treated) resumes, D = 1 for a foreign name.
# Missing values are NOT recoded there: same_exp is NA for every resume in wave 2, and
# reference, accreditation and legal (wave 2 only) are NA for every resume in waves 0-1
# (see the missing-value check below). grf uses missing covariate values directly; the
# tables that need observed values (balance, moderator table, BLP) use waves 0-1 for same_exp.
```

1.4 Setting our ex-ante moderators

We pre-specify three resume characteristics as potential moderators of the immigrant-name penalty, motivated by the hypothesis that credible signals of applicant productivity may reduce employers' reliance on group-based signals (i.e. immigrant-sounding names).

- Language proficiency (language_skills): Language proficiency is an important consideration in employers' hiring of immigrant applicants (Oreopoulos, 2011; Carlsson et al., 2023). It may signal workplace communication abilities and expected productivity, while also serving as an indicator of cultural assimilation (Fossati et al., 2020). Explicit language proficiency signals may therefore reduce employer uncertainty and potentially mitigate the immigrant-name penalty.
- Extracurricular activities (extracurricular_skills): Extracurricular activities (e.g. volunteering) can signal otherwise unobserved soft skills and personal characteristics valued by employers (Alfonso-Costillo et al., 2021), and serve as indicators of cultural assimilation (Fossati et al., 2020), potentially reducing employer uncertainty and mitigating the immigrant-name penalty.
- Multinational firm experience (same_exp): Experience at multinational firms may signal transferable professional skills, including communication and familiarity with established workplace practices (Guillouet et al., 2021). Such signals may reduce employer uncertainty about immigrant applicants' productivity and potentially mitigate the immigrant-name penalty.

```
moderators <- c(
  "language_skills",
  "extracurricular_skills",
  "same_exp"
)
```

1.5 Data transformation

The wave is saved as a factor with three levels (0, 1, 2). It enters X as two dummies, wave_1 and wave_2 (wave 0 is the reference), so a single split can isolate any wave as causal forest does not accept factor variables.

```
df_int1 <- df_int1 %>%
  mutate(
    wave_1 = as.integer(fall_data == 1),
    wave_2 = as.integer(fall_data == 2)
  )

Y_all   <- as.numeric(df_int1$callback)   # outcome: callback
W_all   <- as.numeric(df_int1$D)          # treatment: foreign-sounding name
firm_id <- df_int1$firmid                 # cluster: employer
waves   <- sort(unique(df_int1$fall_data))
```

2 Diagnostic tests

These checks use the data only (no forest). For covariate balance and wave composition see `Causal Forest Balance Table.Rmd`.

2.1 Missing values across waves

The cleaned data keeps missing values as they are, so this shows what is really missing: same_exp is missing for all of wave 2, and reference, accreditation and legal are missing for all of waves 0-1 (they were only on the wave 2 resumes).

```
vars <- c(all_pretreat_cov)

counts_all <- df_int1 %>%
  mutate(across(all_of(vars), as.character)) %>%
  pivot_longer(all_of(vars), names_to = "variable", values_to = "value") %>%
  mutate(variable = factor(variable, levels = vars)) %>%
  group_by(fall_data, variable) %>%
  summarise(
    with_value = sum(!is.na(value) & trimws(value) != ""),
    empty      = sum(is.na(value) | trimws(value) == ""),
    .groups = "drop"
  ) %>%
  left_join(df_int1 %>% count(fall_data, name = "total_rows"), by = "fall_data") %>%
  relocate(total_rows, .after = fall_data) %>%
  arrange(fall_data, variable)

print(counts_all, n = Inf)
```

```
## # A tibble: 21 x 5
##    fall_data total_rows variable               with_value empty
##    <fct>          <int> <fct>                       <int> <int>
##  1 0               1902 language_skills              1902     0
##  2 0               1902 female                       1902     0
##  3 0               1902 ba_quality                   1902     0
##  4 0               1902 extracurricular_skills       1902     0
##  5 0               1902 ma                           1902     0
##  6 0               1902 same_exp                     1902     0
##  7 0               1902 exp_highquality              1902     0
##  8 1               1495 language_skills              1495     0
##  9 1               1495 female                       1495     0
## 10 1               1495 ba_quality                   1495     0
## 11 1               1495 extracurricular_skills       1495     0
## 12 1               1495 ma                           1495     0
## 13 1               1495 same_exp                     1495     0
## 14 1               1495 exp_highquality              1495     0
## 15 2               2260 language_skills              2260     0
## 16 2               2260 female                       2260     0
## 17 2               2260 ba_quality                   2260     0
## 18 2               2260 extracurricular_skills       2260     0
## 19 2               2260 ma                           2260     0
## 20 2               2260 same_exp                        0  2260
## 21 2               2260 exp_highquality              2260     0
```

2.2 Statistics about the data

```r
cat("N:", nrow(df_int1), "(expected 5,657)\n")
```

```
## N: 5657 (expected 5,657)
```

```r
cat("Firms:", length(unique(df_int1$firmid)), "\n")
```

```
## Firms: 3269
```

```r
cat("Resumes per firm:\n")
```

```
## Resumes per firm:
```

```r
print(summary(as.numeric(table(df_int1$firmid))))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    1.00    2.00    1.73    2.00    2.00
```

```r
cat("\nTreatment counts (expected 3,026 control / 2,631 treated):\n")
```

```
##
## Treatment counts (expected 3,026 control / 2,631 treated):
```

```r
print(table(D = df_int1$D))
```

```
## D
##    0    1
## 3026 2631
```

```r
cat("\nName ethnicity by treatment (D must match the coding exactly):\n")
```

```
##
## Name ethnicity by treatment (D must match the coding exactly):
```

```r
print(table(ethnicity = as.character(df_int1$name_ethnicity), D = df_int1$D))
```

```
##            D
## ethnicity     0    1
##   Canada   3026    0
##   Chinese     0  998
##   Indian      0 1167
##   Pakistani   0  466
```

```r
cat("\nWave by treatment:\n")
```

```
##
## Wave by treatment:
```

```r
print(table(wave = df_int1$fall_data, D = df_int1$D))
```

```
##     D
## wave    0    1
##    0  953  949
##    1 1029  466
##    2 1044 1216
```

2.3 Name penalty by wave

Does the name penalty itself differ by wave? Plain regression of callback on D within each wave, so no forest is involved.

- Treatment effect: The coefficient on D measures the difference in callback probability between foreign-named and Canadian-named applicants, expressed in percentage points.
- Clustered standard errors: Uses firm-clustered CR2 standard errors to account for correlation between resumes submitted to the same employer.
- Comparison across waves: Reports the estimated treatment effect, standard error, and 95% confidence interval for each wave.

```r
ate_wave <- bind_rows(lapply(waves, function(k) {
  dat   <- filter(df_int1, fall_data == k)
  dat$y <- as.numeric(dat$callback)
  fit   <- lm_robust(y ~ D, data = dat, clusters = firmid, se_type = "CR2")
  data.frame(wave      = k,
             n         = nrow(dat),
             effect_pp = 100 * unname(coef(fit)["D"]),
             se_pp     = 100 * unname(fit$std.error["D"]),
             lo_pp     = 100 * unname(fit$conf.low["D"]),
             hi_pp     = 100 * unname(fit$conf.high["D"]))
}))

kable(ate_wave, digits = 2,
      caption = "Name penalty by wave (percentage points, firm-clustered CR2)")
```

Table 1: Name penalty by wave (percentage points, firm-clustered CR2)

wave	n	effect_pp	se_pp	lo_pp	hi_pp
0	1902	-4.36	1.17	-6.66	-2.06
1	1495	-4.79	1.36	-7.47	-2.11
2	2260	-4.29	1.11	-6.46	-2.13

2.4 Overlap/Propensity check

We examine the distribution of propensity scores, which estimate the probability of receiving a foreign-sounding name conditional on observed resume characteristics.

```r
# Covariate specifications
propensity_sets <- list(
  "cf1w (3 moderators + wave)" = c(
    moderators,
    "wave_1",
    "wave_2"
  ),
  "cf2 (full covariates)" = c(
    all_pretreat_cov,
    "wave_1",
    "wave_2"
  )
)

# Estimate propensity scores for both specifications
propensity_results <- lapply(propensity_sets, function(vars) {
  set.seed(4101)

  # Construction of the predictor matrix
  X <- df_int1 %>%
    select(all_of(vars)) %>%
    as.matrix()

  # Fit a regression forest to predict treatment assignemnt, rather than callback outcomes.
  propensity_forest <- regression_forest(
    X,
    W_all,
    clusters = firm_id
  )

  # Out-of-bag propensity score predictions
  predict(propensity_forest)$predictions
})

# Observed treatment proportion
p_actual <- mean(W_all)




# Plotting histogram



## Setting axes breaks and maximum
breaks_common <- seq(0, 1, by = 0.025)

y_max <- max(sapply(propensity_results, function(p) {
  max(hist(p, breaks = breaks_common, plot = FALSE)$counts)
}))
```

```r
## Plotting Side-by-side histograms
par(mfrow = c(1, 2))

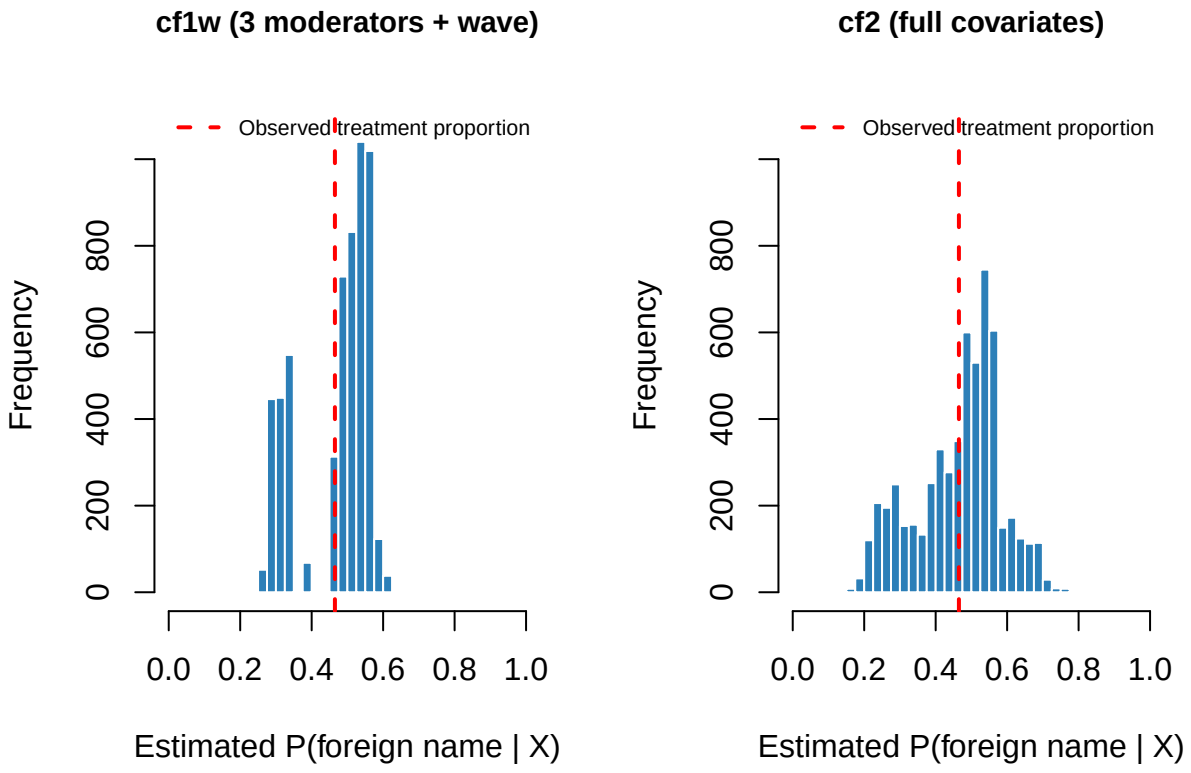
for (nm in names(propensity_results)) {

  W_hat <- propensity_results[[nm]]

  hist(
    W_hat,
    breaks = breaks_common,
    col = "#2c7fb8",
    border = "white",
    main = nm,
    xlab = "Estimated P(foreign name | X)",
    xlim = c(0, 1),
    ylim = c(0, 1.05 * y_max),
    cex.main = 0.9
  )

  abline(
    v = p_actual,
    col = "red",
    lty = 2,
    lwd = 2
  )

  legend(
    "topright",
    legend = "Observed treatment proportion",
    col = "red",
    lty = 2,
    lwd = 2,
    bty = "n",
    cex = 0.7
  )
}
```




```r
par(mfrow = c(1, 1))

# Numerical overlap diagnostics
overlap_summary <- bind_rows(
  lapply(names(propensity_results), function(nm) {

    W_hat <- propensity_results[[nm]]

    data.frame(
      forest = nm,
      observed_treatment = p_actual,
      mean_propensity = mean(W_hat),
      min_propensity = min(W_hat),
      max_propensity = max(W_hat),
      prop_below_0.1 = mean(W_hat < 0.1),
      prop_above_0.9 = mean(W_hat > 0.9)
    )
  })
)

knitr::kable(
  overlap_summary,
  digits = 3,
  caption = "Propensity score overlap diagnostics: cf1w vs cf2"
)
```

Table 2: Propensity score overlap diagnostics: cf1w vs cf2

forest	observed_treatment	mean_propensity	min_propensity	max_propensity	prop_below_0.1	prop_above_0.9
cf1w (3 moderators + wave)	0.465	0.465	0.266	0.606	0	0
cf2 (full covariates)	0.465	0.465	0.157	0.825	0	0

3 Causal forests

3.1 Specifications

Both forests use the same outcome (callback), treatment (D, foreign-sounding name) and clustering (firmid). They differ only in the covariates X:

- **cf1w**: the three moderators plus the wave (two dummies, wave_1 and wave_2), so it is the like-for-like comparison with the wave-adjusted regressions.
- **cf2**: all pre-treatment covariates plus the wave dummies.

The waves differ in design (same_exp is only on the resumes of waves 0-1, and wave 1 uses Pakistani names only), so the forest needs the wave to separate "this moderator matters" from "this wave was different". The wave also enters the forest's propensity (`W.hat`) and outcome (`Y.hat`) models, which absorbs differences in treated share and baseline callback rate across waves. Both forests are validated and reported; neither is picked after the fact. same_exp (NA in wave 2) and reference, accreditation and legal (NA in waves 0-1) are passed to grf as is (grf handles missing covariate values natively).

```
# The wave is included in both forests: the waves differ in design, so the forest needs it
# to separate "this moderator matters" from "this wave was different".

covariate_sets <- list(
  "cf1w (3 moderators + wave)" = c(moderators, "wave_1","wave_2"),
  "cf2 (full covariates)"      = c(all_pretreat_cov, "wave_1","wave_2")
)

# Short labels (cf1w, cf2) for the wide validation tables further down
short_name <- function(x) sub(" .*", "", x)
```

3.2 Fit

Fit causal forests for both specifications (CF1W and CF2). Both use the same outcome (callback), treatment (foreign-sounding name), and employer-level clustering, but differ in their covariates.

```
# function to fit a causal forest for a given set of covariates.
fit_forest <- function(vars) {
  # grf draws its forest seed from R's random number generator, so reset the seed
  # immediately before every fit to make each forest reproducible.
  set.seed(4101)

  # Select the covariates for the current specification and convert to a matrix.
  X <- df_int1 %>% select(all_of(vars)) %>% as.matrix()
```

```
  # Fit a causal forest to estimate heterogeneous treatment effects (CATEs).
  # Y_all: callback outcome (0/1).
  # W_all: foreign-sounding name treatment (0/1).
  # clusters: accounts for dependence among resumes sent to the same employer.
  # grf internally estimates nuisance functions, including propensity scores
  # and expected outcomes, unless these are supplied separately.
  list(cf = causal_forest(X, Y_all, W_all, clusters = firm_id), X = X)
}



# APPLY THE FITTING FUNCTION TO BOTH COVARIATE SPECIFICATIONS
# CF1W: three pre-specified moderators + wave indicators.
# CF2: full set of pre-treatment covariates.
forests <- lapply(covariate_sets, fit_forest)


# Extract the fitted causal forest for subsequent analysis
cf1w   <- forests[["cf1w (3 moderators + wave)"]]$cf
cf2   <- forests[["cf2 (full covariates)"]]$cf

# Save CF2's covariate matrix for subsequent interpretation and diagnostics
X_cf2 <- forests[["cf2 (full covariates)"]]$X
```

4 Causal forest results

4.1 Average treatment effect

Compares the average treatment effect (ATE) of foreign-sounding names on callback probabilities across both causal forest specifications. - ATE estimation: Uses average_treatment_effect() to estimate the overall name penalty for each forest. - Full sample: Sets target.sample = "all" to estimate the ATE across all observations. - Statistical inference: Calculates standard errors and approximate 95% confidence intervals to assess the precision and statistical significance of the estimates. - Comparison across specifications: Examines whether CF1W and CF2 produce similar average treatment effects despite using different sets of covariates.

About `average_treatment_effect` : causal forest uses propensity scores (the estimated probability of receiving treatment) when calculating the Average Treatment Effect (ATE).

```
# Loop through both forest specifications (CF1W and CF2).
ate_tab <- bind_rows(lapply(names(forests), function(nm) {

  # Estimate the ATE using the full sample.
  # grf uses a doubly robust estimator that accounts for
  # estimated propensity scores and expected outcomes.
  a <- average_treatment_effect(forests[[nm]]$cf, target.sample = "all")

  # Extract the ATE estimate and standard error.
  # Multiply by 100 to express results in percentage points.
  data.frame(forest    = nm,
             effect_pp = 100 * a[["estimate"]],
             se_pp     = 100 * a[["std.err"]])
})) %>%
```

```
  # Construct approximate 95% confidence intervals:
  # ATE +/- 1.96 * standard error.
  mutate(lo_pp = effect_pp - 1.96 * se_pp, hi_pp = effect_pp + 1.96 * se_pp)

kable(ate_tab, digits = 3,
      caption = "Average treatment effect by forest (percentage points)")
```

Table 3: Average treatment effect by forest (percentage points)

forest	effect_pp	se_pp	lo_pp	hi_pp
cf1w (3 moderators + wave)	-4.377	0.698	-5.745	-3.009
cf2 (full covariates)	-4.462	0.742	-5.917	-3.006

4.1.1 Interpretation of Causal forest ATE

```
for (i in seq_len(nrow(ate_tab))) {
  cat(sprintf(
    paste0("**Estimated ATE, %s:** having a foreign/ethnic-sounding name changes the ",
           "callback probability by **%.3f** percentage points (SE = %.3f), ",
           "on average across the analysis sample.\n\n"),
    ate_tab$forest[i], ate_tab$effect_pp[i], ate_tab$se_pp[i]
  ))
}
```

Estimated ATE, cf1w (3 moderators + wave): having a foreign/ethnic-sounding name changes the callback probability by **-4.377** percentage points (SE = 0.698), on average across the analysis sample.

Estimated ATE, cf2 (full covariates): having a foreign/ethnic-sounding name changes the callback probability by **-4.462** percentage points (SE = 0.742), on average across the analysis sample.

4.2 Estimated effects (CATE)

The forest's own output: a predicted treatment effect for every resume. Read these as descriptive/exploratory unless the validation section supports heterogeneity. Compare `sd_pp` with `mean_se_pp` in the table below: when the spread of the predictions is no larger than the noise in each prediction, the spread is about what estimation noise alone would produce, so the histograms are not evidence of heterogeneity by themselves.

4.2.1 Distribution of predicted effects

```
# Exploratory distribution of out-of-bag predicted treatment effects.
# These distributions describe forest predictions and do not, by
# themselves, establish statistically significant heterogeneity.

# Obtain out-of-bag CATE predictions from both forests
cate_df <- bind_rows(lapply(names(forests), function(nm) {
  p <- predict(forests[[nm]]$cf, estimate.variance = TRUE)

  data.frame(
```

```
    forest = nm,
    tau_hat = p$predictions,
    tau_se = sqrt(p$variance.estimates)
  )
}))

# Summarise predicted CATE distributions in percentage points
cate_summary <- cate_df %>%
  group_by(forest) %>%
  summarise(
    mean_pp = 100 * mean(tau_hat), # Mean predicted name penalty
    sd_pp = 100 * sd(tau_hat), # Se of predicted effects
    min_pp = 100 * min(tau_hat), # Smallest predicted effect
    p25_pp = 100 * quantile(tau_hat, 0.25), # 25th percentile
    median_pp = 100 * median(tau_hat), # Median predicted effect
    p75_pp = 100 * quantile(tau_hat, 0.75),  # 75th percentile
    max_pp = 100 * max(tau_hat), # Largest predicted effect
    mean_se_pp = 100 * mean(tau_se),  # Average prediction standard error
    .groups = "drop"
  )

# SUmmary statistics for causal forest specification
kable(
  cate_summary,
  digits = 2,
  caption = "Distribution of predicted treatment effects (pp, out-of-bag)"
)
```

Table 4: Distribution of predicted treatment effects (pp, out-of-bag)

forest	mean_pp	sd_pp	min_pp	p25_pp	median_pp	p75_pp	max_pp	mean_se_pp
cf1w (3 moderators + wave)	-4.40	2.94	-11.48	-6.06	-5.08	-1.18	3.69	3.08
cf2 (full covariates)	-4.49	4.67	-18.77	-7.47	-4.47	-1.04	8.51	4.62

```
# Preparing histogram plots and bins


## Convert all predicted treatment effects into percentage points.
## Common bins and axes so the forests can be compared panel by panel
tau_all     <- 100 * cate_df$tau_hat

# Create histogram bins of width 1 percentage point,
# covering the full range of predictions from both forests.
cate_breaks <- seq(floor(min(tau_all)), ceiling(max(tau_all)), by = 1)

# Determine a common y-axis limit based on the largest bin count.
# This prevents differences in plot scaling from distorting comparisons.
cate_ylim   <- c(0, 1.05 * max(sapply(names(forests), function(nm) {
  max(hist(100 * cate_df$tau_hat[cate_df$forest == nm],
           breaks = cate_breaks, plot = FALSE)$counts)
```

```
}))))



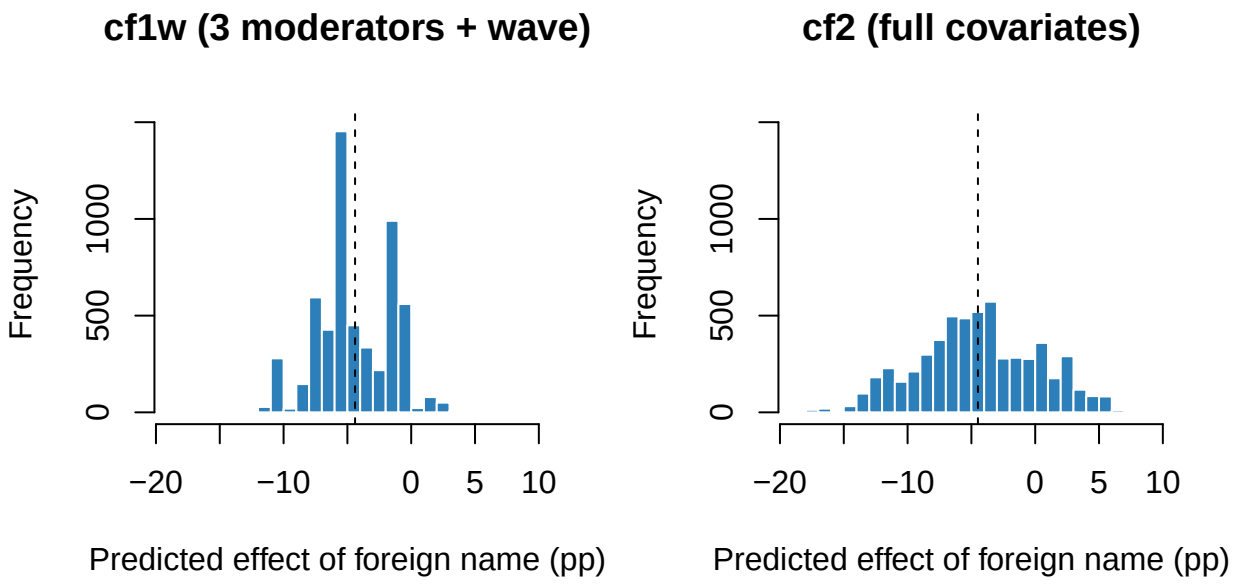

## Display the histograms side by side, one for each forest.
par(mfrow = c(1, length(forests)))


## Plotting histograms for each of the forests
for (nm in names(forests)) {

  # Extract predicted CATEs for the current forest.
  tau_pp <- 100 * cate_df$tau_hat[
    cate_df$forest == nm
  ]

  # Plot the distribution of predicted name penalties.
  # Common x- and y-axis limits allow direct comparison across forests.
  hist(
    tau_pp,
    breaks = cate_breaks,
    col = "#2c7fb8",
    border = "white",
    main = nm,
    xlab = "Predicted effect of foreign name (pp)",
    xlim = range(cate_breaks),
    ylim = cate_ylim
  )

  # Dashed line indicates mean predicted CATE
  abline(
    v = mean(tau_pp),
    lty = 2
  )
}
```




```
par(mfrow = c(1, 1))
```

4.2.2 Confidence Interval of CATEs

This chunk examines whether the estimated name penalty varies across different resume characteristics using the full causal forest specification (CF2).

```
# Remaining pre-treatment covariates + wave controls
other_covariates <- c(
  setdiff(all_pretreat_cov, moderators),
  "wave_1",
  "wave_2"
)

# Combine the pre-specified moderators and remaining covariates.
# These variables will be used to define subgroups for CF2.
cf2_vars <- c(moderators, other_covariates)

# Estimate subgroup Average Treatment Effects (ATEs) using CF2.
# Each subgroup is defined by a particular value of a covariate.
subgroup_results <- lapply(cf2_vars, function(v) {


  # Identify all observed, non-missing values of the covariate.
  # For binary variables, these are typically 0 and 1.
  values <- sort(unique(na.omit(df_int1[[v]])))

  bind_rows(lapply(values, function(val) {


    # Identify observations belonging to the current subgroup.
```

```
    # Exclude observations with missing values for this covariate.
    idx <- which(
      !is.na(df_int1[[v]]) &
      df_int1[[v]] == val
    )


    # Estimate the ATE within the selected subgroup.
    # uses the fitted CF2 forest and its doubly robust estimator.
    # same setting as above: target.sample = "all" estimates the ATE over the selected subgroup.
    ate <- average_treatment_effect(
      cf2,
      subset = idx,
      target.sample = "all"
    )

    data.frame(
      covariate = v,
      value = as.character(val),
      n = length(idx),
      effect_pp = 100 * ate[["estimate"]],
      se_pp = 100 * ate[["std.err"]]
    )
  }))
})



# Construct 95% confidence intervals for subgroup results
subgroup_df <- bind_rows(subgroup_results) %>%
  mutate(
    lower = effect_pp - 1.96 * se_pp,
    upper = effect_pp + 1.96 * se_pp,
    group_label = value
  )
```

4.3 Subgroup plots

4.3.1 Plotting function for subgroups

```
## Common plotting function
plot_subgroup_ci <- function(data, vars, title, ncol = 3) {


  # Select the requested covariates and preserve their order.
  plot_data <- data %>%
    filter(covariate %in% vars) %>%
    mutate(
      covariate = factor(covariate, levels = vars)
    )

  ggplot(
```

```
    plot_data,
    aes(x = group_label, y = effect_pp)
  ) +

    geom_hline(
      yintercept = 0,
      linetype = "dashed",
      color = "grey50"
    ) +

    geom_errorbar(
      aes(ymin = lower, ymax = upper),
      width = 0.15,
      color = "#2c7fb8",
      linewidth = 0.8
    ) +

    geom_point(
      color = "#2c7fb8",
      size = 2.5
    ) +

    facet_wrap(
      ~ covariate,
      ncol = ncol,
      axes = "all_x",
      axis.labels = "all_x"
    ) +

    labs(
      x = "Covariate value (non-missing observations)",
      y = "Estimated treatment effect (percentage points)",
      title = title,
      subtitle = "cf2: Subgroup ATEs with 95% confidence intervals"
    ) +

    theme_minimal(base_size = 11) +

    theme(
      strip.text = element_text(face = "bold"),
      plot.title = element_text(face = "bold"),
      axis.text.x = element_text(size = 9),
      panel.spacing = grid::unit(1.2, "lines")
    )
}
```

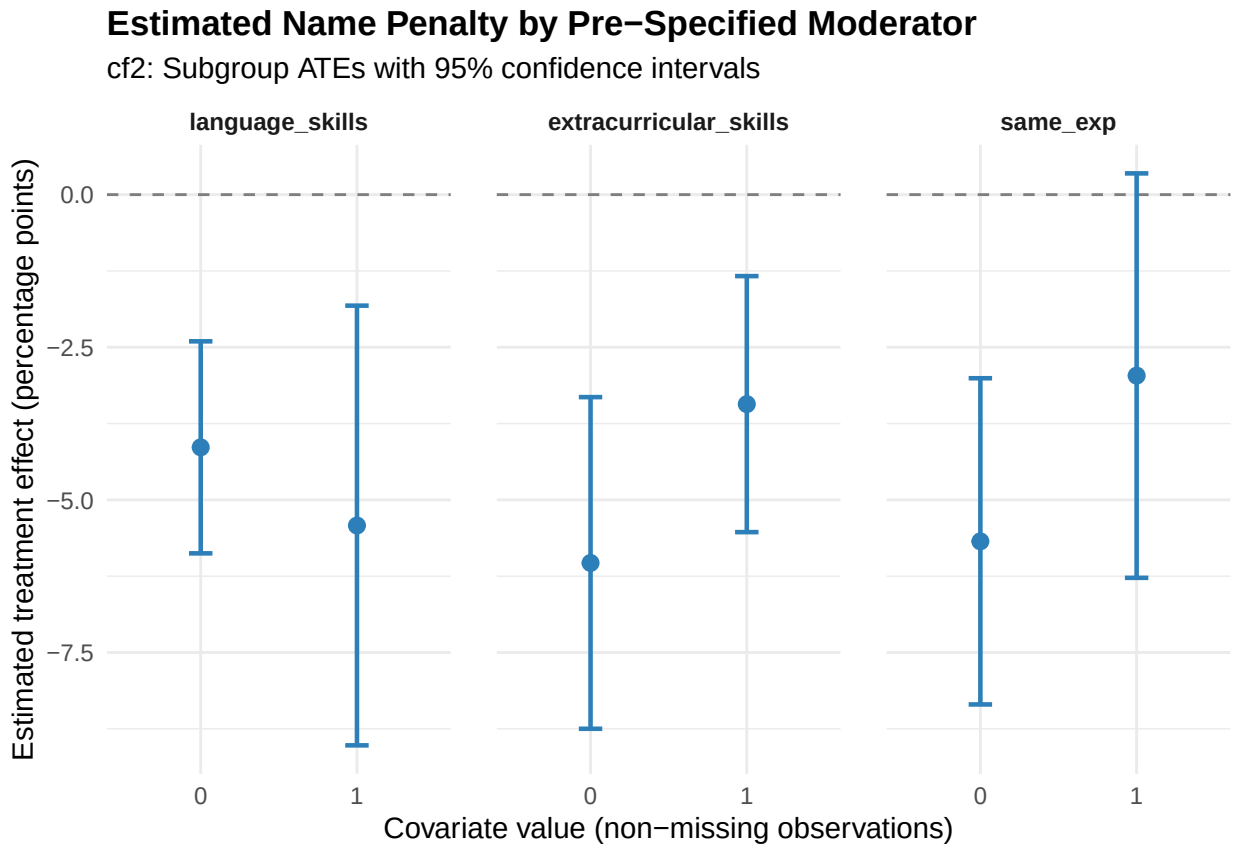
4.3.2 GRAPH 1: Plot subgroup ATEs for the three pre-specified moderators.

The graph compares the estimated name penalty between resumes with and without language skills, extracurricular skills, and matching work experience.

```
# Compare the estimated name penalty across different values
# of language skills, extracurricular skills, and same experience.

plot_moderators <- plot_subgroup_ci(
  subgroup_df,
  moderators,
  "Estimated Name Penalty by Pre-Specified Moderator",
  ncol = 3
)

print(plot_moderators)
```



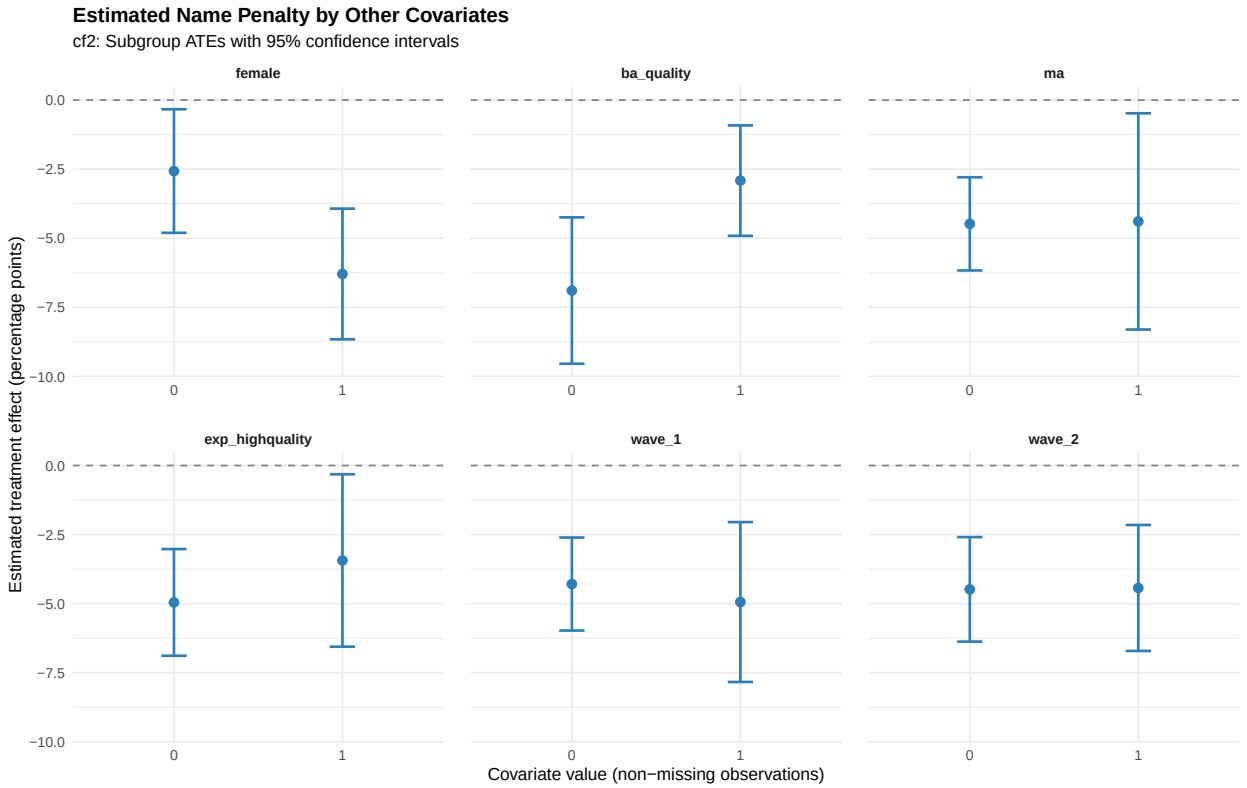
4.3.3 GRAPH 2: Other pre-treatment covariates

This examines whether the estimated name penalty varies across the remaining pre-treatment resume characteristics and experimental wave indicators using CF2.

```
# Plot subgroup ATEs for each of the remaining pre-treatment covariates and
# experimental wave indicators using the previously defined function.
plot_other <- plot_subgroup_ci(
  subgroup_df,
  other_covariates,
  "Estimated Name Penalty by Other Covariates",
  ncol = 3
```

```
)

# Plots graph
print(plot_other)
```




```
# Numerical results for all subgroups, including 3 moderators
knitr::kable(
  subgroup_df %>%
    select(covariate, value, n, effect_pp, se_pp, lower, upper),
  digits = 3,
  caption = "cf2 subgroup ATEs with 95% confidence intervals (pp)"
)
```

Table 5: cf2 subgroup ATEs with 95% confidence intervals (pp)

covariate	value	n	effect_pp	se_pp	lower	upper
language_skills	0	4233	-4.139	0.886	-5.876	-2.403
language_skills	1	1424	-5.420	1.837	-9.021	-1.819
extracurricular_skills	0	2240	-6.033	1.386	-8.749	-3.317
extracurricular_skills	1	3417	-3.432	1.070	-5.528	-1.335
same_exp	0	1899	-5.679	1.363	-8.351	-3.007
same_exp	1	1498	-2.964	1.690	-6.277	0.348
female	0	2781	-2.568	1.141	-4.804	-0.332
female	1	2876	-6.293	1.207	-8.658	-3.928
ba_quality	0	2200	-6.893	1.351	-9.541	-4.244
ba_quality	1	3457	-2.915	1.019	-4.913	-0.917

covariate	value	n	effect_pp	se_pp	lower	upper
ma	0	4515	-4.480	0.861	-6.166	-2.793
ma	1	1142	-4.391	1.996	-8.304	-0.477
exp_highquality	0	3820	-4.955	0.985	-6.885	-3.025
exp_highquality	1	1837	-3.436	1.592	-6.556	-0.317
wave_1	0	4162	-4.290	0.859	-5.973	-2.607
wave_1	1	1495	-4.940	1.476	-7.834	-2.047
wave_2	0	3397	-4.482	0.965	-6.373	-2.591
wave_2	1	2260	-4.431	1.163	-6.710	-2.152

4.3.4 Best linear projection (cf2, three moderators)

The BLP regresses cf2's doubly robust scores on the three moderators (`A`). The other covariates still enter through cf2's nuisance estimates; `A` only sets what the effect is summarised on. The coefficients are a joint linear approximation: they do not hold the other covariates fixed, and they need not match the one-at-a-time interaction terms above when the moderators are correlated. Resumes where any moderator is missing are excluded with `subset`, so this uses waves 0-1 (same_exp is missing in wave 2). The intercept is the effect when all three moderators are 0.

```
# Extract the three moderators from CF2's covariate matrix.
# These are language skills, extracurricular skills, and same experience.
A_mod  <- X_cf2[, moderators, drop = FALSE]


# Restrict the analysis to resumes with all three moderators observed.
#  excludes resumes with missing moderator values,
# particularly wave 2 where same_exp is unavailable.
blp_ok <- which(complete.cases(A_mod))

# Report the number of resumes and unique employers used in the BLP.
cat("Resumes used:", length(blp_ok),
    "| firms:", length(unique(firm_id[blp_ok])), "\n\n")
```

```
## Resumes used: 3397 | firms: 1927
```

```
# Estimate the Best Linear Projection of CF2 treatment effects
# onto the three pre-specified moderators.
# Each coefficient measures the association between a moderator
# and the estimated name penalty, holding other moderators constant.
blp <- best_linear_projection(cf2, A_mod, subset = blp_ok)
print(blp)
```

```
##
## Best linear projection of the conditional average treatment effect.
## Confidence intervals are cluster- and heteroskedasticity-robust (HC3):
##
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)            -0.071030   0.021704 -3.2726 0.001076 **
## language_skills         0.028948   0.030799  0.9399 0.347335
## extracurricular_skills  0.013510   0.025341  0.5331 0.593987
## same_exp                0.027662   0.023606  1.1718 0.241361
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.3.5 Predicted effect by covariate cell (cf1w)

cf1w has only a few distinct covariate combinations, so show them directly (same_exp = NA is wave 2). It examines how the predicted name penalty varies across different combinations of the three pre-specified moderators using CF1W.

```
cf1w_cells <- df_int1 %>%
  mutate(tau_hat = predict(cf1w)$predictions) %>%
  group_by(language_skills, extracurricular_skills, same_exp) %>%
  summarise(n = n(), effect_pp = 100 * mean(tau_hat), .groups = "drop") %>%
  arrange(effect_pp)

kable(cf1w_cells, digits = 2, caption = "cf1w: predicted effect by covariate cell (pp)")
```

Table 6: cf1w: predicted effect by covariate cell (pp)

language_skills	extracurricular_skills	same_exp	n	effect_pp
1	0	NA	308	-10.72
1	0	0	162	-7.00
0	0	0	597	-6.88
0	0	NA	571	-5.21
1	1	NA	422	-5.04
0	1	0	900	-4.83
0	0	1	494	-4.81
1	1	1	184	-4.03
0	1	1	712	-3.28
1	1	0	240	-2.01
0	1	NA	959	-1.10
1	0	1	108	0.96

5 Validation of causal forest results

5.1 test_calibration

The differential forest prediction is the number that matters. grf's p-value on it is one-sided (is the coefficient above zero?). Near 1 means well calibrated, between 0 and 1 means the forest overstates the spread, and at or below 0 means the spread isn't tracking real differences.

```
calibration <- lapply(forests, function(f) test_calibration(f$cf))

for (nm in names(calibration)) {
  cat("\n====", nm, "====\n")
  print(calibration[[nm]])
}
```

```
##
## ==== cf1w (3 moderators + wave) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
```

```
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction          0.99453    0.15804  6.2931 1.673e-10 ***
## differential.forest.prediction -0.18527    0.30052 -0.6165    0.7312
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## ==== cf2 (full covariates) ====
##
## Best linear fit using forest predictions (on held-out data)
## as well as the mean forest prediction as regressors, along
## with one-sided heteroskedasticity-robust (HC3) SEs:
##
##                                Estimate Std. Error t value    Pr(>t)
## mean.forest.prediction         0.994295   0.157222  6.3241 1.371e-10 ***
## differential.forest.prediction 0.021829   0.189598  0.1151    0.4542
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cal_tab <- bind_rows(lapply(names(calibration), function(nm) {
  tc <- calibration[[nm]]
  data.frame(
    forest           = nm,
    mean_est         = tc["mean.forest.prediction", "Estimate"],
    mean_se          = tc["mean.forest.prediction", "Std. Error"],
    diff_est         = tc["differential.forest.prediction", "Estimate"],
    diff_se          = tc["differential.forest.prediction", "Std. Error"],
    diff_p_one_sided = tc["differential.forest.prediction", "Pr(>t)"]
  )
}))

kable(cal_tab, digits = 3, caption = "test_calibration results")
```

Table 7: test_calibration results

forest	mean_est	mean_se	diff_est	diff_se	diff_p_one_sided
cf1w (3 moderators + wave)	0.995	0.158	-0.185	0.301	0.731
cf2 (full covariates)	0.994	0.157	0.022	0.190	0.454

5.2 RATE / TOC with a firm-level split

Half of the firms train a forest that produces the priority scores; the other half trains a separate evaluation forest. The split is by firm, not by resume, so resumes from one employer never sit on both sides. AUTOC is the primary measure and QINI the secondary one, chosen in advance. The priority score is the forest's predicted effect with no sign flip, so the top of the ranking is the group with the smallest predicted penalty (the least negative effects), and a positive RATE means the ranking sorts correctly.

Seed 1 is the headline split, fixed in advance. Seeds 2 to `n_seeds` are stability checks only. This chunk fits many forests and can take a few minutes.

```r
n_seeds <- 5

make_split <- function(firm, seed) {
  set.seed(seed)    # set immediately before sampling so the split is reproducible
  f <- unique(firm)
  firm %in% sample(f, floor(length(f) / 2))
}

# Train on one half of the firms and evaluate on the other half.
# `targets` picks the RATE measures to compute (one list element each: autoc, qini).
# `...` passes extra arguments (e.g. tuning settings) to causal_forest.
# The RATE standard errors are bootstrapped, so computing extra targets uses up
# random draws; the tuning check asks for AUTOC only so its draws stay unchanged.
run_rate <- function(X, Y, W, firm, train, seed, targets = c("AUTOC", "QINI"), ...) {
  cf_train <- causal_forest(X[train, , drop = FALSE],  Y[train],  W[train],
                            clusters = firm[train],  seed = seed, ...)
  cf_eval  <- causal_forest(X[!train, , drop = FALSE], Y[!train], W[!train],
                            clusters = firm[!train], seed = seed, ...)
  priorities <- predict(cf_train, X[!train, , drop = FALSE])$predictions
  rates <- lapply(targets, function(tg) {
    rank_average_treatment_effect(cf_eval, priorities, target = tg)
  })
  setNames(rates, tolower(targets))
}

rate_res <- lapply(forests, function(f) {
  out <- lapply(seq_len(n_seeds), function(s) {
    train <- make_split(firm_id, s)
    run_rate(f$X, Y_all, W_all, firm_id, train, seed = s)
  })
  names(out) <- seq_len(n_seeds)
  out
})

rate_tab <- bind_rows(lapply(names(rate_res), function(nm) {
  bind_rows(lapply(names(rate_res[[nm]]), function(s) {
    a <- rate_res[[nm]][[s]]$autoc
    q <- rate_res[[nm]][[s]]$qini
    data.frame(forest = nm, seed = as.integer(s), headline = (s == "1"),
               autoc = a$estimate, autoc_se = a$std.err,
               autoc_lo = a$estimate - 1.96 * a$std.err,
               autoc_hi = a$estimate + 1.96 * a$std.err,
               qini = q$estimate, qini_se = q$std.err)
  }))
}))

rate_tab %>%
  mutate(forest = short_name(forest)) %>%
  kable(digits = 4, caption = "RATE by forest and split seed (headline = seed 1)")
```

Table 8: RATE by forest and split seed (headline = seed 1)

forest	seed	headline	autoc	autoc_se	autoc_lo	autoc_hi	qini	qini_se
cf1w	1	TRUE	-0.0063	0.0102	-0.0262	0.0137	-0.0027	0.0036
cf1w	2	FALSE	-0.0082	0.0100	-0.0278	0.0114	-0.0012	0.0032
cf1w	3	FALSE	0.0004	0.0107	-0.0206	0.0214	-0.0016	0.0035
cf1w	4	FALSE	0.0155	0.0104	-0.0049	0.0360	0.0063	0.0033
cf1w	5	FALSE	-0.0086	0.0110	-0.0302	0.0130	-0.0011	0.0034
cf2	1	TRUE	-0.0063	0.0132	-0.0323	0.0197	-0.0027	0.0039
cf2	2	FALSE	-0.0123	0.0111	-0.0341	0.0094	-0.0048	0.0031
cf2	3	FALSE	0.0088	0.0103	-0.0114	0.0290	0.0043	0.0036
cf2	4	FALSE	0.0115	0.0117	-0.0114	0.0344	0.0029	0.0033
cf2	5	FALSE	-0.0051	0.0127	-0.0299	0.0198	-0.0003	0.0036

TOC curves for the headline split. The three-moderator forest (cf1w) can produce only a few distinct predictions, so many resumes are tied in the ranking and its curve will look stepped; mention this in the write-up.

```
library(patchwork)

fig_col <- c(cf1w = "#E69F00", cf2 = "#0072B2")

# Panel A: AUTOC by firm split, with 95% intervals (percentage points)
rate_plot <- rate_tab %>%
  mutate(forest = short_name(forest),
         x      = seed + ifelse(forest == "cf1w", -0.12, 0.12))

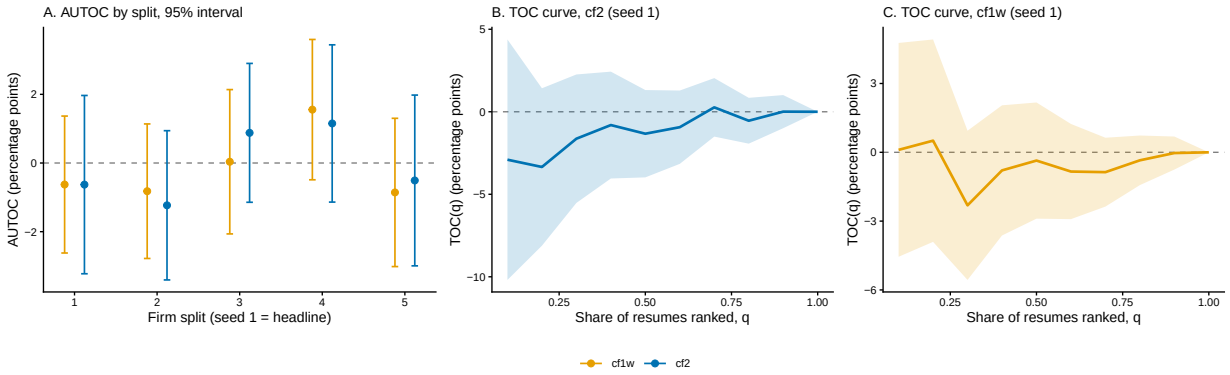
panel_a <- ggplot(rate_plot, aes(x = x, y = 100 * autoc, colour = forest)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
  geom_errorbar(aes(ymin = 100 * autoc_lo, ymax = 100 * autoc_hi),
                width = 0.08, linewidth = 0.5) +
  geom_point(size = 1.8) +
  scale_colour_manual(values = fig_col, breaks = c("cf1w", "cf2"), name = NULL) +
  scale_x_continuous(breaks = 1:n_seeds) +
  labs(title = "A. AUTOC by split, 95% interval",
       x = "Firm split (seed 1 = headline)", y = "AUTOC (percentage points)") +
  theme_classic(base_size = 9) +
  theme(plot.title = element_text(size = 9, hjust = 0))

# Panels B and C: TOC curve for the headline split (seed 1) with 95% bands.
# rank_average_treatment_effect() stores the curve in $TOC (columns estimate, std.err, q).
toc_panel <- function(nm, title) {
  toc <- as.data.frame(rate_res[[nm]][["1"]]$autoc$TOC)
  col <- fig_col[[short_name(nm)]]
  ggplot(toc, aes(x = q)) +
    geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50") +
    geom_ribbon(aes(ymin = 100 * (estimate - 1.96 * std.err),
                    ymax = 100 * (estimate + 1.96 * std.err)),
                fill = col, alpha = 0.18) +
    geom_line(aes(y = 100 * estimate), colour = col, linewidth = 0.8) +
    labs(title = title, x = "Share of resumes ranked, q",
         y = "TOC(q) (percentage points)") +
    theme_classic(base_size = 9) +
```

```
    theme(plot.title = element_text(size = 9, hjust = 0))
}

panel_b <- toc_panel("cf2 (full covariates)",      "B. TOC curve, cf2 (seed 1)")
panel_c <- toc_panel("cf1w (3 moderators + wave)", "C. TOC curve, cf1w (seed 1)")

figure1 <- (panel_a + panel_b + panel_c) +
  plot_layout(widths = c(1.15, 1, 1), guides = "collect") &
  theme(legend.position = "bottom")
print(figure1)
```




```
# Save a copy for the report
dir.create("figures", showWarnings = FALSE)
ggsave("figures/figure1_rate.png", figure1, width = 11, height = 3.6, dpi = 300)
```

5.3 Combined reading (rule fixed in advance)

Both checks support heterogeneity: calibration differential coefficient positive and significant (one-sided $p < 0.05$) and the headline AUTOC interval excluding 0. Neither supports it: report that we find no statistically reliable evidence of heterogeneity, and do not interpret forest subgroup patterns. They disagree: call the evidence inconclusive and treat any subgroup patterns as exploratory only. A null here may reflect low power, not constant effects. If the verdict is a null, covariate means across held-out priority groups are not reported: that descriptive step is only meaningful when the RATE is clearly non-zero.

```
verdict_tab <- cal_tab %>%
  left_join(filter(rate_tab, headline), by = "forest") %>%
  mutate(
    calibration_supports = diff_p_one_sided < 0.05,
    rate_supports        = autoc_lo > 0,
    verdict = case_when(
      calibration_supports & rate_supports   ~ "Both support heterogeneity",
      !calibration_supports & !rate_supports ~ "Neither supports: report null",
      TRUE                                   ~ "Disagree: inconclusive"
    )
  ) %>%
  select(forest, diff_est, diff_se, diff_p_one_sided,
         autoc, autoc_lo, autoc_hi, verdict)
```

```
verdict_tab %>%
  mutate(forest = short_name(forest)) %>%
  kable(digits = 3, caption = "Validation verdict by forest",
        col.names = c("Forest", "Diff. est.", "Diff. SE", "p (1-sided)",
                      "AUTOC", "AUTOC lo", "AUTOC hi", "Verdict"))
```

Table 9: Validation verdict by forest

Forest	Diff. est.	Diff. SE	p (1-sided)	AUTOC	AUTOC lo	AUTOC hi	Verdict
cf1w	-0.185	0.301	0.731	-0.006	-0.026	0.014	Neither supports: report null
cf2	0.022	0.190	0.454	-0.006	-0.032	0.020	Neither supports: report null

5.4 Tuning robustness

One alternative setting, fixed in advance: twice the default trees (4000 vs 2000) and a larger honesty fraction (0.7 vs the grf default 0.5). It is not a grid. The question is only whether the null verdict changes. Seed 1 is the same firm split as the headline RATE result, so the comparison is like for like.

```
alt <- list(num.trees = 4000, honesty.fraction = 0.7)

# Calibration under the alternative setting
forests_alt <- lapply(forests, function(f) {
  causal_forest(f$X, Y_all, W_all,
                clusters         = firm_id,
                num.trees        = alt$num.trees,
                honesty.fraction = alt$honesty.fraction,
                seed             = 123)
})

cal_alt <- bind_rows(lapply(names(forests_alt), function(nm) {
  tc <- test_calibration(forests_alt[[nm]])
  data.frame(
    forest           = nm,
    diff_est         = tc["differential.forest.prediction", "Estimate"],
    diff_se          = tc["differential.forest.prediction", "Std. Error"],
    diff_p_one_sided = tc["differential.forest.prediction", "Pr(>t)"]
  )
}))

# AUTOC under the alternative setting (headline split, seed 1)
train_1 <- make_split(firm_id, 1)

rate_alt_tab <- bind_rows(lapply(names(forests), function(nm) {
  a <- run_rate(forests[[nm]]$X, Y_all, W_all, firm_id, train_1, seed = 1,
                targets          = "AUTOC",
                num.trees        = alt$num.trees,
                honesty.fraction = alt$honesty.fraction)$autoc
  data.frame(forest   = nm,
```

```
             autoc    = a$estimate,
             autoc_lo = a$estimate - 1.96 * a$std.err,
             autoc_hi = a$estimate + 1.96 * a$std.err)
}))

# Default vs alternative
stab_tab <- cal_tab %>%
  select(forest, diff_default = diff_est, p_default = diff_p_one_sided) %>%
  left_join(select(cal_alt, forest, diff_alt = diff_est, p_alt = diff_p_one_sided),
            by = "forest") %>%
  left_join(filter(rate_tab, headline) %>%
              select(forest, autoc_default = autoc,
                     lo_default = autoc_lo, hi_default = autoc_hi),
            by = "forest") %>%
  left_join(select(rate_alt_tab, forest, autoc_alt = autoc,
                   lo_alt = autoc_lo, hi_alt = autoc_hi),
            by = "forest")

stab_tab %>%
  mutate(forest = short_name(forest)) %>%
  kable(digits = 3,
        caption = "Default vs alternative tuning (4000 trees, honesty fraction 0.7)",
        col.names = c("Forest", "Diff (def)", "p (def)", "Diff (alt)", "p (alt)",
                      "AUTOC (def)", "Lo (def)", "Hi (def)",
                      "AUTOC (alt)", "Lo (alt)", "Hi (alt)"))
```

Table 10: Default vs alternative tuning (4000 trees, honesty fraction 0.7)

Forest	Diff (def)	p (def)	Diff (alt)	p (alt)	AUTOC (def)	Lo (def)	Hi (def)	AUTOC (alt)	Lo (alt)	Hi (alt)
cf1w	-0.185	0.731	-0.186	0.737	-0.006	-0.026	0.014	-0.007	-0.026	0.013
cf2	0.022	0.454	0.035	0.416	-0.006	-0.032	0.020	-0.010	-0.036	0.016

Reading it: the verdict is stable if every forest still has a calibration p above 0.05 and an AUTOC interval that includes 0 under the alternative setting. If one forest flips, report it as sensitive to tuning and treat the evidence for that forest as inconclusive under your combined rule.